



Autonomous Data Analyst Agent

Complete Technical Specification, Source Code, System Flowcharts & Deployment Blueprint

Status: LIVE IN PRODUCTION

Platform: Streamlit Community Cloud

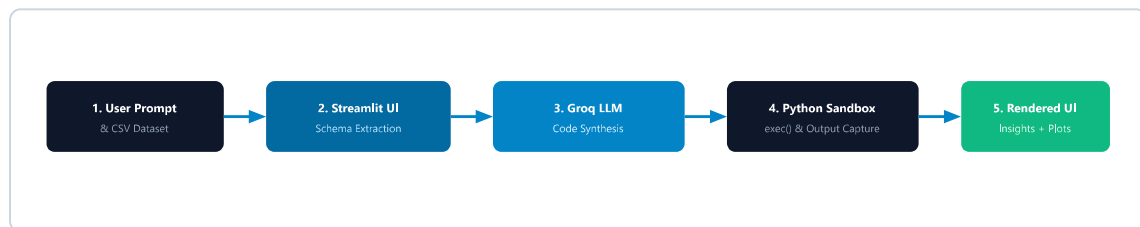
Live URL: <https://autonomous-data-analyst-507.streamlit.app>

GitHub: [singamalaomsai/autonomous-data-analyst-agent](https://github.com/singamalaomsai/autonomous-data-analyst-agent)

1. System Architecture & Component Responsibilities

Layer	Technology	Core Function
Frontend UI	Streamlit 1.32+	Multi-turn conversational chat, CSV data loader, dynamic chart and table rendering.
LLM Engine	Groq API (llama-3.3-70b-versatile)	Query intent parsing, dataset schema reasoning, and Python code generation.
Execution Sandbox	Python exec() + contextlib	Isolated runtime executing Pandas code with captured stdout and figure streams.
Visualization	Matplotlib & Seaborn	Automatic chart generation (bar charts, line graphs, distributions) for UI rendering.
Cloud Hosting	Streamlit Community Cloud	Containerized Linux runtime linked directly to GitHub main branch with encrypted secrets.

2. End-to-End System Execution Flowchart



3. Complete Source Code Implementation

3.1 Dependencies (requirements.txt)

```
streamlit>=1.32.0
pandas>=2.0.0
numpy>=1.24.0
matplotlib>=3.7.0
```

```
seaborn>=0.12.0
groq>=0.5.0
```

3.2 Git Exclusions (.gitignore)

```
.venv/
venv/
__pycache__/
*.py[cod]
.streamlit/secrets.toml
.env
```

3.3 Tool Execution Engine & Sandbox (analyst_tools.py)

```
import io
import sys
import contextlib
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

def get_dataset_overview(df: pd.DataFrame) -> dict:
    buffer = io.StringIO()
    df.info(buf=buffer)
    info_str = buffer.getvalue()

    return {
        "columns": list(df.columns),
        "shape": df.shape,
        "dtypes": {k: str(v) for k, v in df.dtypes.items()},
        "missing_values": df.isnull().sum().to_dict(),
        "info_summary": info_str,
        "head": df.head(5).to_dict(orient="records")
    }

def execute_python_code(code_string: str, df: pd.DataFrame) -> dict:
    stdout_capture = io.StringIO()
    fig_created = False
    plt.close('all')

    local_vars = {
        "df": df.copy(),
        "pd": pd,
        "np": np,
        "plt": plt,
        "sns": sns
    }

    try:
        with contextlib.redirect_stdout(stdout_capture):
            exec(code_string, {}, local_vars)

        output_text = stdout_capture.getvalue()

        if plt.get_fignums():
            fig_created = True

        return {
            "success": True,
            "output": output_text.strip() if output_text.strip() else "Code executed successfully with no print output.",
            "fig": plt.gcf() if fig_created else None
        }
```

```

    }
except Exception as e:
    return {
        "success": False,
        "output": f"Runtime Execution Error: {str(e)}",
        "fig": None
    }

```

3.4 Streamlit Web Application (app.py)

```

import os
import streamlit as st
import pandas as pd
from groq import Groq
from analyst_tools import get_dataset_overview, execute_python_code

st.set_page_config(
    page_title="Autonomous Data Analyst Agent",
    page_icon="📊",
    layout="wide"
)

api_key = st.secrets.get("GROQ_API_KEY", os.getenv("GROQ_API_KEY"))

if not api_key:
    st.error("⚠️ Groq API Key not detected. Please configure GROQ_API_KEY in Secrets.")
    st.stop()

client = Groq(api_key=api_key)

st.title("📊 Autonomous Data Analyst Agent")
st.caption("Powered by Groq LLM, Pandas, & Tool-Calling Runtime Execution")

st.sidebar.header("📁 Data Source")
uploaded_file = st.sidebar.file_uploader("Upload CSV Dataset", type=["csv"])

if uploaded_file is not None:
    df = pd.read_csv(uploaded_file)
    st.sidebar.success(f"Loaded: {uploaded_file.name} ({df.shape[0]} rows, {df.shape[1]} cols)")
else:
    default_path = "company_metrics.csv"
    if os.path.exists(default_path):
        df = pd.read_csv(default_path)
        st.sidebar.info(f"Loaded default: {default_path}")
    else:
        st.sidebar.warning("Please upload a CSV dataset to begin.")
        df = None

if "messages" not in st.session_state:
    st.session_state.messages = []

for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])
        if "figure" in msg and msg["figure"] is not None:
            st.pyplot(msg["figure"])

if prompt := st.chat_input("Ask a question about the dataset..."):
    if df is None:
        st.warning("Please upload or select a CSV dataset first.")
        st.stop()

    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

```

```

with st.chat_message("assistant"):
    overview = get_dataset_overview(df)

    system_prompt = f"""
You are an expert Autonomous Data Analyst Agent.
Dataset Overview:
- Columns: {overview['columns']}
- Data Types: {overview['dtypes']}
- Shape: {overview['shape']}
- Sample Preview: {overview['head']}

Instructions:
1. Formulate a rigorous analytical strategy to answer the user request.
2. Write clean, executable Python code operating strictly on the df DataFrame.
3. Use print() to display numeric/text findings or plt/sns to generate visualizations.
4. Wrap all executable code in a single python ... block.
"""

    response = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": prompt}
        ],
        temperature=0.2
    )

    agent_reply = response.choices[0].message.content
    st.markdown(agent_reply)

    fig = None
    if "python" in agent_reply:
        code_block = agent_reply.split("python")[1].split("```")[0].strip()
        with st.expander("🔍 View Agent Executed Code"):
            st.code(code_block, language="python")

        result = execute_python_code(code_block, df)
        if result["success"]:
            if result["output"]:
                st.success("Execution Output:")
                st.text(result["output"])
            if result["fig"]:
                fig = result["fig"]
                st.pyplot(fig)
        else:
            st.error(result["output"])

    st.session_state.messages.append({
        "role": "assistant",
        "content": agent_reply,
        "figure": fig
    })

```

4. Troubleshooting & Authentication Resolution

Issue: Git 403 Forbidden Access

Windows Credential Manager cached an old GitHub identity. Resolved by clearing generic credentials for git:
[\[https://github.com\]](https://github.com) (<https://github.com>) and binding username explicitly:

```
git remote set-url origin [https://singamalaomsai@github.com/singamalaomsai/autonomous-data-analyst-agent.git](https://singamalaomsai@github.com/singamalaomsai/autonomous-data-analyst-ag
```

```
ent.git)
git push -u origin main
```

Streamlit Cloud Production Secrets: Injected GROQ_API_KEY directly in Streamlit Cloud Dashboard under Advanced Settings to keep tokens secure and gitignored.

5. Verification Matrix

Verification Vector	Methodology	Production Result
Mobile Responsiveness	Accessed live URL on mobile Chrome & Safari	Clean UI layout with interactive chat and chart rendering.
Autonomous Execution	Prompted: "Analyze department revenue and plot bar chart"	Agent generated code, executed analytics, and rendered chart live.
Security Isolation	Inspected public GitHub repository	Secrets completely protected via gitignore isolation.